GPU streams: overlapping copies and kernels

[M2 DiPaC] GPU Programming, Université Paris-Saclay – Session 7

Oguz Kaya

Assistant Professor
Université Paris-Saclay and LISN, Gif-sur-Yvette, France

université PARIS-SACLAY

Objectives

- Hide the **copies** between the CPU and the GPU behind the computation
- Know which operations are **asynchronous** – they return before the GPU is done – and how the CPU waits for the GPU
- Use **pinned** host memory, the condition for asynchronous copies
- Use **streams**, queues of GPU operations that can run at the same time, and **pipeline** a computation over them
- Avoid the **synchronisations** that silently undo the overlap

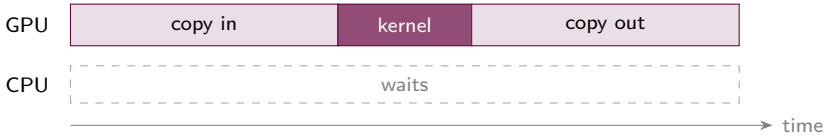
Outline

1. The idea: overlapping copies and kernels
2. Asynchronous operations
3. Streams
4. What undoes the overlap
5. Summary

Outline

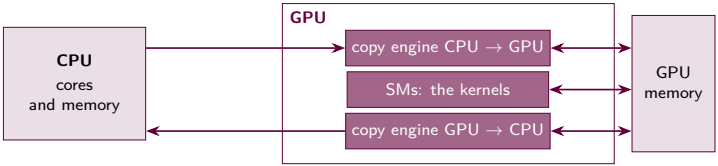
1. The idea: overlapping copies and kernels
2. Asynchronous operations
3. Streams
4. What undoes the overlap
5. Summary

A GPU program so far: copy in, kernel, copy out



- The programs of the previous sessions copy the input to the GPU, run the kernel and copy the result back, **one after the other**, while the CPU waits
- The link between the CPU and the GPU is much slower than the GPU's own memory: for many programs the **copies take longer than the kernel**, and a faster kernel changes little
- The copies have to be **hidden** behind the computation: this is the subject of this session

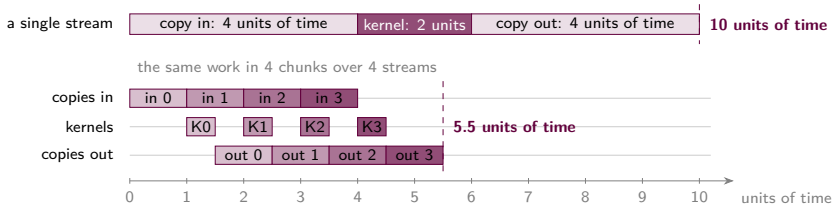
The GPU can copy and compute at the same time



separate hardware: the two copy engines, the SMs and the CPU cores can all work at the same time, independently

- The GPU has two **copy engines**, one for each direction: separate hardware that runs the copies in the background while the **SMs** run kernels and the **CPU cores** do other work, independently
- This gives a potential **4-way parallelism** to exploit: copies in, kernels, copies out and CPU work at the same time
- The program of the previous slide uses one of them at a time

The pipeline, in principle



- One after the other, as in a single stream: the copy in takes 4 units of time, the kernel 2 and the copy out 4, so **10 units of time**
- In 4 chunks, chunk 1 comes in while chunk 0 is computed, and so on: the copy engines and the SMs work at the same time, **5.5 units of time**
- What it takes, built step by step: operations that **return at once** (asynchronous), and **streams** to say which operations are independent

Outline

1. The idea: overlapping copies and kernels
2. Asynchronous operations
3. Streams
4. What undoes the overlap
5. Summary

Step 1. The synchronous program

The running example, `y = f(x)` on `n` floats (all the sizes are multiples of 256):

```
__host__ __device__ float f(float v);   // the same f on the CPU and the GPU
__global__ void work(const float *x, float *y, int n)
{
  int i = blockIdx.x * blockDim.x + threadIdx.x;
  if (i < n) { y[i] = f(x[i]); }
}
```

The program, with `hx`, `hy` on the CPU and `dx`, `dy` on the GPU:

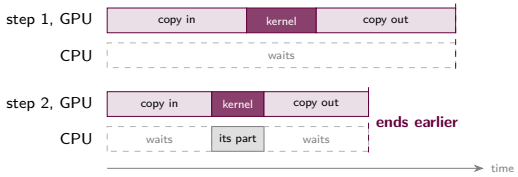
```
1 size_t bytes = n * sizeof(float);
2 cudaMemcpy(dx, hx, bytes, cudaMemcpyHostToDevice);
3 work<<<n / 256, 256>>>(dx, dy, n);
4 cudaMemcpy(hy, dy, bytes, cudaMemcpyDeviceToHost);
```

line	call	for the CPU
2	cudaMemcpy	**synchronous**: the CPU waits until the copy is done
3	kernel launch	**asynchronous**: returns at once, the kernel runs on the GPU
4	cudaMemcpy	**synchronous**: starts after the kernel, the CPU waits until it is done

Step 2. The CPU takes part of the work

The same kernel on the first ng elements only (ng $\leq$ n); the CPU computes the others meanwhile:

```
1 cudaMemcpy(dx, hx, ng * sizeof(float), cudaMemcpyHostToDevice);
2 work<<<ng / 256, 256>>>(dx, dy, ng);                  // returns at once
3 for (int i = ng; i < n; i++) { hy[i] = f(hx[i]); }    // meanwhile, on the CPU
4 cudaMemcpy(hy, dy, ng * sizeof(float), cudaMemcpyDeviceToHost);
```



- The launch of line 2 returns at once, so the loop of line 3 runs on the CPU **while the GPU computes**; line 4 then waits for the kernel
- Doing some of the work on the CPU **saves GPU time** on all three: less to copy in, to compute and to copy out

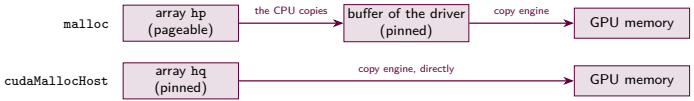
Step 3. cudaDeviceSynchronize()


```
1 double t0 = now();                    // now(): the host clock
2 work<<<n / 256, 256>>>(dx, dy, n);
3 double t1 = now();                    // t1 - t0: only the launch
4 cudaDeviceSynchronize();              // the CPU waits until the kernel is done
5 double t2 = now();                    // t2 - t0: the whole kernel
```

- A launch returns at once: `t1 - t0` measures the launch, not the kernel
- `cudaDeviceSynchronize()` makes the CPU **wait** until all the GPU work issued so far is done, so `t2 - t0` is the time of the kernel
- Use it whenever the CPU must be sure that the GPU has finished: to time a kernel, or before using results that no `cudaMemcpy` waited for

Step 4. Pinned host memory


```
1 float *hp = (float *)malloc(bytes);     // pageable
2 float *hq;
3 cudaMallocHost(&hq, bytes);             // pinned (page-locked)
4 ...                                     // hq is used like any array
5 cudaFreeHost(hq);
```

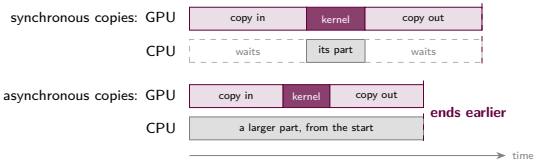


- `malloc` memory is **pageable**: the system may move it. The copy engine reads only memory that stays in place, so the driver first copies a pageable array into a pinned buffer of its own
- **Pinned** memory is read directly by the copy engine: faster, and the copy can be **asynchronous** (next step)
- Allocate it once, only for the arrays that are copied: it is slow to allocate

Step 5. cudaMemcpyAsync: a copy that returns at once

The example of step 2 with asynchronous copies. cudaMemcpyAsync takes the same arguments as cudaMemcpy (plus a stream, next section), queues the copy and returns at once. Here **hx and hy must be pinned** (cudaMallocHost): from pageable memory the copy would not be asynchronous.

```
1 cudaMemcpyAsync(dx, hx, ng * sizeof(float), cudaMemcpyHostToDevice);
2 work<<<ng / 256, 256>>>(dx, dy, ng);
3 cudaMemcpyAsync(hy, dy, ng * sizeof(float), cudaMemcpyDeviceToHost);
4 for (int i = ng; i < n; i++) { hy[i] = f(hx[i]); }     // the CPU starts right away
5 cudaDeviceSynchronize();                               // waits for the GPU's part
```



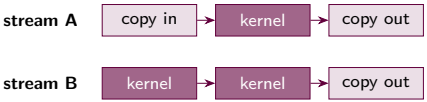
- Lines 1–3 return at once: the CPU work **starts much earlier** and overlaps the copies and the kernel, so the CPU can take a larger part and the GPU does less
- Until line 5, do not overwrite hx, nor read the GPU's part of hy

Outline

1. The idea: overlapping copies and kernels
2. Asynchronous operations
3. Streams
4. What undoes the overlap
5. Summary

Step 6. A stream: create, use, destroy

```
1 cudaStream_t s;              // a stream: a queue of GPU operations
2 cudaStreamCreate(&s);        // create it, once
3 ...                          // queue operations in s (next slides)
4 cudaStreamDestroy(s);        // destroy it at the end
```



in a stream: **in order**, the next operation starts when the previous one is done

between streams: **no order** – they can overlap

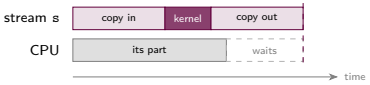
the CPU issues the operations and goes on

- The operations of one stream run **in the order** they were issued; the operations of different streams may run **at the same time**
- An operation given no stream goes to the **default stream**: every program so far used it

Step 7. The example of step 5, in a stream

The GPU's part now goes to a stream s, declared, created and destroyed by the program:

```
1 cudaStream_t s;                                                    // a stream
2 cudaStreamCreate(&s);                                              // create it
3 cudaMemcpyAsync(dx, hx, ng * sizeof(float), cudaMemcpyHostToDevice, s);
4 work<<<ng / 256, 256, 0, s>>>(dx, dy, ng);
5 cudaMemcpyAsync(hy, dy, ng * sizeof(float), cudaMemcpyDeviceToHost, s);
6 for (int i = ng; i < n; i++) { hy[i] = f(hx[i]); }                 // meanwhile, the CPU's part
7 cudaStreamSynchronize(s);                                          // waits for the operations of s
8 cudaStreamDestroy(s);                                              // destroy it
```

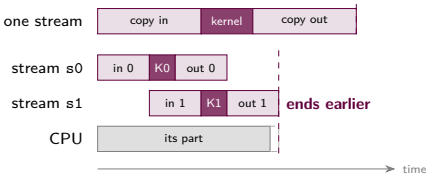


- The stream is the **last argument** of `cudaMemcpyAsync` and the **4th launch parameter** (the 3rd, dynamic shared memory, is 0)
- `cudaStreamSynchronize(s)` waits for `s` only. In one stream, the GPU's operations still run **one after the other**

Step 8. Two streams, one half of the GPU's part each

The CPU queues all the GPU work first, half in s0 and half in s1, then does its own part:

```
1  cudaStream_t s0, s1;  cudaStreamCreate(&s0);  cudaStreamCreate(&s1);
2  int h = ng / 2;  size_t hb = h * sizeof(float);          // the GPU's part, in two halves
3  cudaMemcpyAsync(dx, hx, hb, cudaMemcpyHostToDevice, s0);
4  work<<<h / 256, 256, 0, s0>>>(dx, dy, h);
5  cudaMemcpyAsync(hy, dy, hb, cudaMemcpyDeviceToHost, s0);
6  cudaMemcpyAsync(dx + h, hx + h, hb, cudaMemcpyHostToDevice, s1);
7  work<<<h / 256, 256, 0, s1>>>(dx + h, dy + h, h);
8  cudaMemcpyAsync(hy + h, dy + h, hb, cudaMemcpyDeviceToHost, s1);
9  for (int i = ng; i < n; i++) { hy[i] = f(hx[i]); }       // then the CPU's part
10 cudaStreamSynchronize(s0);  cudaStreamSynchronize(s1);
11 cudaStreamDestroy(s0);  cudaStreamDestroy(s1);
```

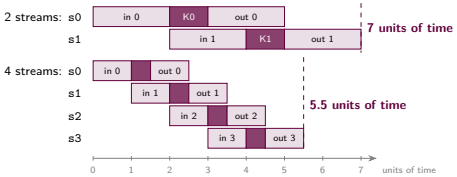


- **No order** between the streams: the copy in of half 1 runs during the kernel and copy out of half 0
- The GPU's part ends earlier

Step 9. K chunks over S streams

université PARIS-SACLAY

```
1  cudaStream_t s[S];  for (int j = 0; j < S; j++) { cudaStreamCreate(&s[j]); }
2  int c = ng / K;  size_t cb = c * sizeof(float);          // the GPU's part, in K chunks
3  for (int k = 0; k < K; k++) {
4    int o = k * c;  cudaStream_t sk = s[k % S];            // chunk k goes to stream k % S
5    cudaMemcpyAsync(dx + o, hx + o, cb, cudaMemcpyHostToDevice, sk);
6    work<<<c / 256, 256, 0, sk>>>(dx + o, dy + o, c);
7    cudaMemcpyAsync(hy + o, dy + o, cb, cudaMemcpyDeviceToHost, sk);
8  }
9  for (int i = ng; i < n; i++) { hy[i] = f(hx[i]); }       // the CPU's part
10 cudaDeviceSynchronize();                                 // waits for all the streams
11 for (int j = 0; j < S; j++) { cudaStreamDestroy(s[j]); }
```

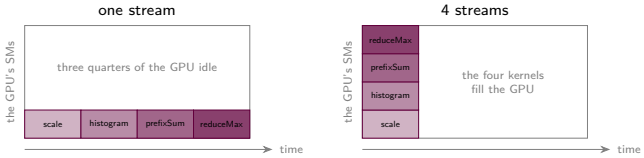


- With 4 chunks in **4 streams** instead of 2 in 2, the pipeline fills and empties faster: 5.5 units of time instead of 7
- The streams are created once and reused in turn; with $S = 1$ the chunks gain nothing

Step 10. Small kernels in different streams run together

Four different, independent kernels, each launched with only 4 blocks:

```
1 scale<<<4, 256, 0, s[0]>>>(x0, n0);              // four small tasks,
2 histogram<<<4, 256, 0, s[1]>>>(x1, n1, hist);    // independent of each other
3 prefixSum<<<4, 256, 0, s[2]>>>(x2, n2);
4 reduceMax<<<4, 256, 0, s[3]>>>(x3, n3, xmax);
5 cudaDeviceSynchronize();
```



- Each kernel uses a small part of the GPU, and they compute different things: they cannot be merged into one launch
- In one stream they run one after the other; in four streams they **run together** and fill the GPU

Outline

1. The idea: overlapping copies and kernels
2. Asynchronous operations
3. Streams
4. What undoes the overlap
5. Summary

Step 11. The default stream

The loop of step 9, with one stream argument forgotten:

```
1 for (int k = 0; k < K; k++) {
2   int o = k * c;  cudaStream_t sk = s[k % S];
3   cudaMemcpyAsync(dx + o, hx + o, cb, cudaMemcpyHostToDevice, sk);
4   work<<<c / 256, 256, 0, sk>>>(dx + o, dy + o, c);
5   cudaMemcpyAsync(hy + o, dy + o, cb, cudaMemcpyDeviceToHost);  // no stream!
6 }
```

- Line 5 has **no stream**: the copy goes to the **default stream**, like every launch without a 4th parameter, every `cudaMemcpy` and every `cudaMemset`
- On the GPU, an operation of the default stream is a **barrier**: it starts only when everything issued before it in the other streams is done, and what is issued after it in the other streams waits for its end. The CPU is not stopped (that would be `cudaDeviceSynchronize()`); the GPU serialises
- The chunks run one after the other again: in the loop, every operation must get its chunk's stream, never the default one

Step 12. Calls that synchronise the whole GPU

The loop of step 9, with a GPU buffer allocated for each chunk:

```
1 for (int k = 0; k < K; k++) {
2   int o = k * c;  cudaStream_t sk = s[k % S];
3   float *t;  cudaMalloc(&t, cb);                          // a buffer for the chunk
4   cudaMemcpyAsync(t, hx + o, cb, cudaMemcpyHostToDevice, sk);
5   work<<<c / 256, 256, 0, sk>>>(t, dy + o, c);
6   cudaMemcpyAsync(hy + o, dy + o, cb, cudaMemcpyDeviceToHost, sk);
7   cudaFree(t);
8 }
```

These calls synchronise the GPU, or keep the streams from overlapping:

- `cudaMalloc`, `cudaFree`
- `cudaMallocHost`, `cudaHostAlloc`, `cudaFreeHost`
- `cudaMemcpy`, `cudaMemset` (without `Async`), and the copies from GPU to GPU memory
- any operation in the **default stream**
- `cudaDeviceSetCacheConfig` (the L1 / shared memory split)
- `cudaDeviceSynchronize()`

Line 7 is correct only *because* `cudaFree` waits for the GPU, and that wait kills the pipeline: **allocate before** the loop, **free after** it.

What it takes to overlap

- **Pinned** host memory for the arrays that are copied, allocated once
- **Independent chunks**: each with its own part of the arrays
- **Every operation in a stream**: `cudaMemcpyAsync(..., s)` and `<<<grid, block, 0, s>>>`
- **Nothing that waits** inside the loop: no `cudaMemcpy`, no default stream, no allocation, no free
- **Wait once**, at the end – or stream by stream, when the CPU can start on part of the result
- **Chunks large enough**: every operation costs some microseconds before it starts; thousands of tiny chunks pay it thousands of times

Outline

1. The idea: overlapping copies and kernels
2. Asynchronous operations
3. Streams
4. What undoes the overlap
5. Summary

What to remember


```
cudaMallocHost(&h, bytes);    // pinned
cudaFreeHost(h);

cudaStream_t s;               // a stream
cudaStreamCreate(&s);
cudaStreamSynchronize(s);     // wait for s
cudaStreamDestroy(s);

// queued in s:
cudaMemcpyAsync(dst, src, bytes, kind, s);
kernel<<<grid, block, 0, s>>>(args);

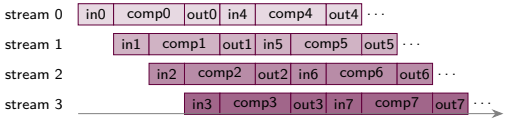
cudaDeviceSynchronize();      // wait for all
```

- Launches, and `cudaMemcpyAsync` from **pinned** memory, **return at once**; the CPU waits only when told to
- A **stream** runs its operations in order; different streams overlap
- Chunks over a few streams: the copies of one chunk overlap the kernel of another
- One call in the **default stream**, or an allocation in the loop, undoes it all

This week's lab (1/2): column by column



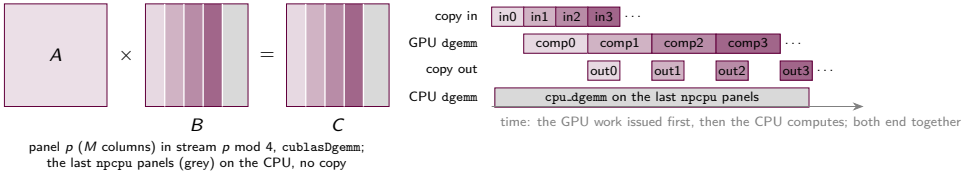
column j: 8 KB in, `matvec`, 8 KB out, in stream j mod 4 (the shades)



Exercise 1, single precision, $C = AB$ with $n = 2048$, column-major: one **column** of B in, a matrix-vector kernel, one column of C out

1. your `matvec` kernel (one thread per row), and the **sequential** loop: `cudaMemcpy`, kernel, `cudaMemcpy`
2. S **streams** and S slots on the GPU: column j in stream j mod S, `cudaMemcpyAsync` from pinned memory; $S = 1, 2, 4, 8, 16$, then pageable memory

This week's lab (2/2): panels, cuBLAS and the CPU



panel p (M columns) in stream p mod 4, `cublasDgemm`; the last `npcpu` panels (grey) on the CPU, no copy

Exercise 2, double precision, $n = 1536$: **panels** of M columns

3. panel p in stream p mod 4: `cudaMemcpyAsync`, **`cublasDgemm`** in that stream (`cublasSetStream`), `cudaMemcpyAsync`; $M = 8 \ldots 512$
4. the **CPU** joins in: the last `npcpu` panels with the given `cpu_dgemm` (AVX2, all the cores), issued *after* the GPU work – **four things at once**

Double precision: the GPU is 32–64 times slower there than in single precision, the CPU only 2 times – so the CPU is worth a real share. Build with `-lcublas -Xcompiler -fopenmp -lgomp`; then the mini-exam.

Contact

Oguz Kaya
Université Paris-Saclay and LISN, Gif-sur-Yvette, France
oguz.kaya@universite-paris-saclay.fr
www.oguzkaya.com

Course material: `github.com/kayaogz/2627-m2dipac-gpuprogramming`